1. An overview/executive summary of your project (a paragraph or two).

Our project, Daedalus' Tower, is a 3D, first person puzzle game, wherein the player is trapped within a tower themed after the mythological Labyrinth. The floors of the tower rotate, unlocking areas of the tower and allowing for puzzle solving. The player's goal is to ascend through the floors of the tower, completing puzzles themed around Greek gods: Poseidon themed water puzzles and Zeus themed electricity puzzles specifically. Once the player finishes climbing to the highest floor of the tower, they will reach a final puzzle, forging a sword to place in a statue of the Minotaur on the lowest level, which will release wings so the player can escape the tower and win the game.

The highlights of the project that make it stand out amongst other video games are the tower rotation mechanics, the unique puzzles incorporating the rotation of the tower tiers, and the story following the Greek legend, Daedalus. Combining all of these traits will make an uncommon and immersive game that will be a fun experience for the player.

2. A concise statement of the MVP. This is one to two sentences.

The MVP is a game including a fully functional tower that has rotation mechanics and engaging water and electricity themed puzzles that allow the player to ascend the tower. Once the player reaches the top of the tower, they allow for completion of an end state puzzle, and complete the game.

3. The current team (you!) with short two sentence bios as well as what "role" / lead you are each taking in the project.

- Olivia: I am mainly responsible for the implementation of the game/puzzle mechanics. I am more of a backend developer with little front end for this project, since Nathanael and Will have the most artistic ability and creativity of the group.

- Nathanael: I am responsible for the puzzle ideas and a main creative developer. I came up with the puzzle ideas and the game idea as well as being part of the theming talks. Besides being one of the lead creative developers, I am also going to work on the implementation of the code, assets, and mechanics to make sure everything fits together properly.

- Will: I am responsible for most of the 3D assets and visual look of the game, including designing all the assets, shaders, particles, and implementation of the visual effects on the game. This also includes creation of the game map and implementation of the puzzle

design in 3D space. I will be responsible for incorporating the puzzles into the 3D environment.

4. A description and/or list of the functional requirements of your end product and/or minimum viable product (what must the final or MVP be able to do?).

Character System:

- Character movement:
  Basic player movement. This should be physics driven to best interact with the water system. Essentially adding force to a rigidbody to move in the direction associated with the camera. This only handles basic WASD movement, which should be suitable up to the point of ladder/climbing to higher levels. This will be from a first person perspective, so all player transforms will be handled with this in mind.
- Minimap:
  The player will be able to press a button, i.e. TAB that pulls up a smaller version of the whole Tower from an outside perspective. This should contain markers for floor differentiation, the player position, and include the ability to transform some floor levels through the mini-map. This will also disable player movement and pause any ongoing tower rotation while the map is active. This will be tied to the movement of the full scale tower and keep a constant marker of the player on the map.
    - Dependencies: Tower, tower movement, player movement, and surrounding environment
- Interaction System:
  The player will need to interact with the environment in three main ways: Placing markers in the environment, rotating the environment, and interacting with puzzles, such as by pressing buttons. All of this will be handled by the same system, pressing ‘E’ to interact with an interactable, which will then react based on its own script. It is essential that the player is able to properly place markers as they make their way through the tower, as disorientation will likely be a factor when the tower rotates. It is also important that the player is able to interact with all of the puzzles, as the puzzles would be impossible to complete without this feature.
    - Dependencies: Assets, Puzzle Areas, Interaction Points, Puzzle Design, Minimap

Environment system:

- Assets:
  This refers to a very broad set of 3D models that will be designed from scratch for the project. These models should all be modular, so they can be used by different puzzles. All of the models must have a cohesive look, be designed to work well together, and be minimal poly-count for optimization. Environmental Assets, Puzzle Assets, UI assets,

and Interactable assets are the main categories to be done. Everything in these categories must be completed and textured using Substance Painter.

- Requirements/resources: Blender/Substance Painter

- Animation:
  There will be some animation that must be done to handle various interactions in game. These will mostly be used for particle effects when items are placed, the tower is rotated, and puzzle visual effects, such as water splashing, electricity arcing, and smoky shadows. Anything not solid would usually have an element of animation to it when the player interacts with objects, we should be able to see that etc.
  - Requirements/Resources: Blender/Unity particle system
- Puzzles:
  This refers to basic puzzle design. Puzzles must be designed to fit the tower shape, and all potential states for the puzzle must be known. These puzzles can suit whatever shape, but they must utilize at least one of the following systems: tower rotation, water puzzles, and electricity puzzles. The combination of these three systems allows for many different types, scales, and complexity of puzzles. Making sure each puzzle unlocks the ability for another to be completed and so on is the main difficulty here.

  The rotation puzzles will function in combination with the other two sets of puzzles, while also lining up terrain in the map to allow for traversal vertically. The water puzzles will primarily be based on water flow. As the tower is rotated, water sources fill and empty based on the state of the tower. Most of this will be directly downward flow, for the sake of constraining scope. The electricity puzzles will primarily be based on beam-splitting. Splitting and recombining beams of electricity to match certain power requirements, powering certain devices to trigger tower rotation, movement of objects in game, etc.
  - Dependencies: Assets
- Tower movement:
  The tower movement will need to be handled from multiple states, both when the player triggers rotation, and when a certain stage of the puzzle is reached. This will be done through scripting a simple rotation script held on each level of the tower. By handling it with a simple script, we can branch out and use it to create larger and larger structures to scale up once our Minimum Viable Product is met. This rotation must freeze player movement, rotate one or more levels of the tower at once, and be set up in such a way that the puzzles move properly with the tower rotation.
  - Dependencies: Tower assets, Player movement
  - Requirements/resources: No known requirements/resources
- Map:
  The map itself must be designed, and should be done based on the puzzle design and done in separate levels. Each tier of the tower needs to be designed on its own while still

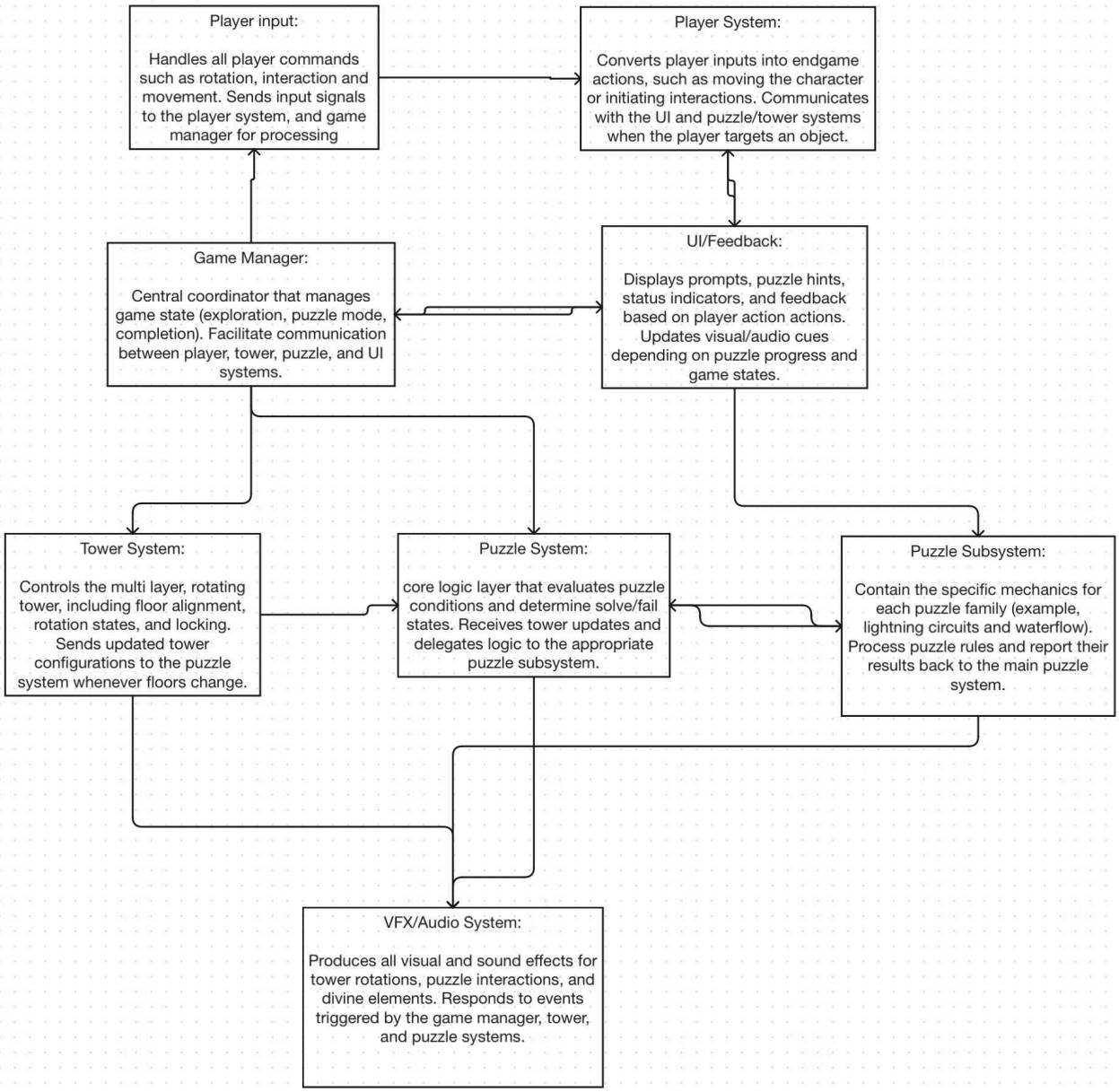
providing a fully cohesive whole. This involves placement of features, knowing how each tier can rotate, and the ability to interact with the other tiers. We know that the map will have a central tower containing a large body of water that fills and empties based on the orientation of the map, thereby allowing the player to change what level they are on. We also know this will involve blocking water flow points, and points of connectivity for the electricity puzzles.

- Constraints: Potential learning curves with designing a fully functional map
- Requirements/resources: Blender/Substance Painter

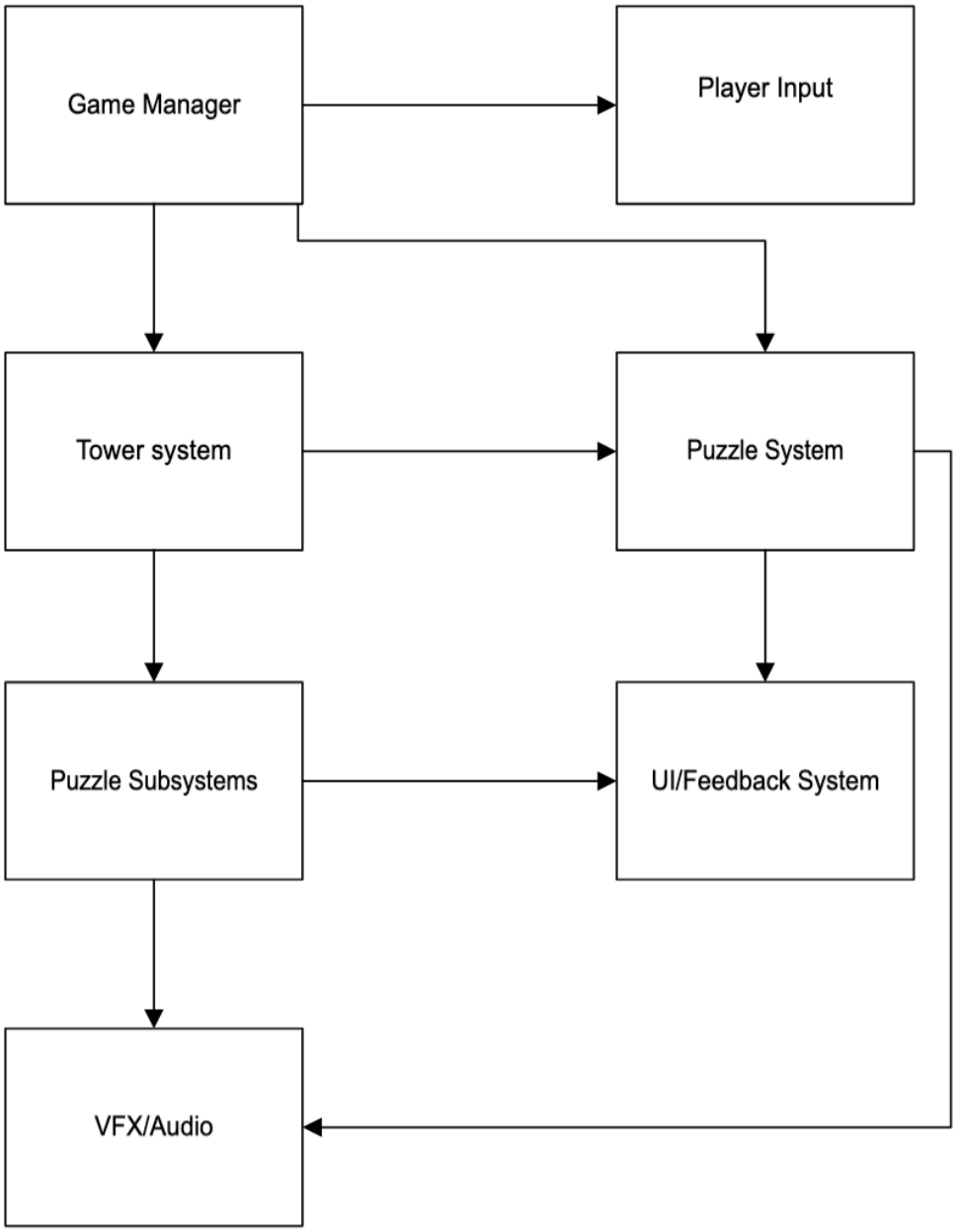
- Render texture downscaling implementation:
  The visual style of this game will be similar to that of the game Devil Daggers. This means that each asset will have fully scaled textures that will be reduced in scale and then projected over the model to give the appearance of a lower visual fidelity while preserving model quality. This will give the appearance of a lower bit screen, keeping textures consistent in scale in proportion with the distance of objects to the screen.
  - Constraints: Learning curve
  - Requirements/resources: Camera targets texture rendering in lower resolution then amplified
- Color design:
  Coming up with a constant visual feel of the game will require selecting a good color pallet. This will help with the creation of physical designs and create a consistent mood for everything in the game to avoid the clashing visual feel that some games can create. Our initial color palette consists primarily of oranges and blues, which connect with the electricity and water puzzles respectively, as well as just looking really good with each other.
- Water System:
  One of the primary puzzle components of this game is a fully functioning water system. This system can be most easily done by using quads with shaders to emulate water, and using raycasting from “sources” of water directly downward, detecting when the tower is in a position that allows for the water to flow, then connecting with the source quad and raising it according to a flow rate variable. This will allow us to use the tower position itself as an answer to if the puzzle is solved, but visually emulate the water flowing.
- Electricity System:
  The other primary puzzle component here is a system emulating electricity. This will primarily be by conducting “lightning” in the form of a particle system by connecting conductive materials, and “splitting” the current into multiple beams of electricity which power different things on the tower and which require traversal and rotation of the tower to be able to solve. This will mostly be done by storing tower rotation and changing the visuals as the tower is rotated.

- Sound Effects and Sound Design:
  All these other features are all well and good, but without good sound, no game is ever complete. This includes interaction sound effects, massive grinding sounds for rotation, water flow sound effects, electrical sounds, mechanical noises, puzzle complete sounds, footsteps, stabbing (for the end sequence) etc. These sounds will all have to be sourced or made from scratch, tweaked to match the game, then implemented into the game.

5. A High-Level Systems Overview (e.g. what are the major components in your system); Diagrams and Text descriptions would be appropriate.



6. A High-Level Block diagram showing the major components and sub components of the project and how they fit together.



7. List and Detailed Description of the MAJOR Milestones for your project (think top 3-4 things) – How will you know when you have successfully reached/completed each milestone?

Completion of the game map - Once all elements of the game map have been placed, including the assets for puzzles in their correct orientation. This will showcase the completion of all assets and puzzle design being completed. At this point, only the mechanics within the game will need to be implemented, and the environment gives us a fantastic playground for this. This includes the full implementation of the level rotation mechanics.

Water puzzles implemented - Once all water physics and puzzle components have been implemented we will have half of the puzzle components and a good chunk of the level traversal completed. After this combined with the tower implementation from the other milestone, most of the core gameplay will be implemented.

Electricity puzzles implemented - Once all electricity puzzles and beam splitting mechanics have been implemented, we will have all of the core puzzle components and traversal completed. At this point the game is fully playable but lacking an ending.

Ending implemented - With the puzzles implemented, we must implement the system that allows for the forging of the sword and the end phase of the game. Without this, the game functions, but without an end state. By creating the trigger for the completion of the sword and opening of the final gate, the game will have an end state for this demo stage of the game, opening a door to future possibilities for future extension as well.

8. A List of major tasks that lead to each milestone (no details required); think in segments of ~2 week tasks for one person; set and assign the task to a lead for now (it might change later). This may be from Trello or another planning site, not required, but would be really helpful.

- **You should capture the above as both a task listing as well as some graphical timeline (gantt chart?) representation to show relationships (if any) between tasks.**

<u>Completion of game map:</u>

- Asset Creation (Will)
- Color design (Will)
- Mini-Map (Olivia)
- Puzzle layout (Nathanael)
- Map Design (Nathanael)
- Interaction (Nathanael)
- Level rotation mechanics (Olivia)
- Character movement (Will)
- Basic Animation (Will)
- Basic Sound Effect Implementation (Will)

Water puzzles implemented:

- Water physics implementation (Olivia)
- Water shaders and particle design (Will)
- Puzzle components/functionality added to map (Nathanael)
- Water puzzle implementation (Nathanael)
- Hint system (Olivia)
- Water and puzzle sounds (Will)

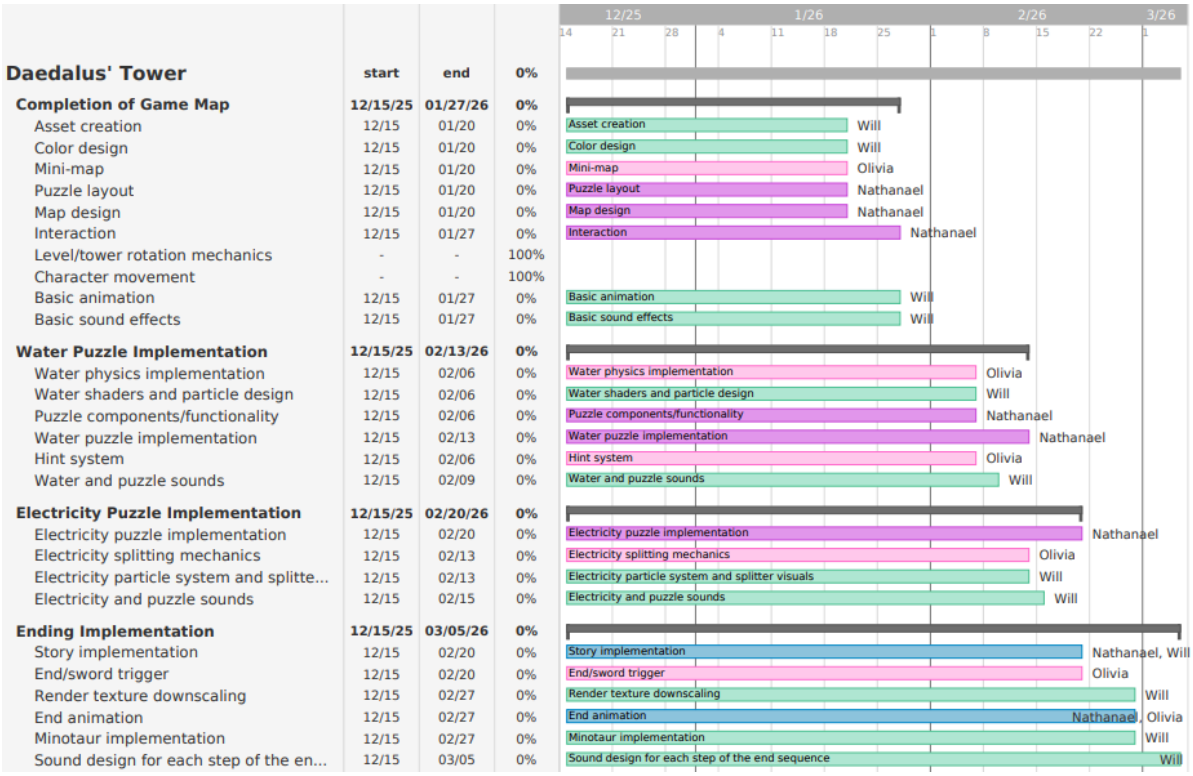
Electricity puzzles implemented:

- Electricity puzzle implementation (Nathanael)
- Electricity splitting mechanics (Olivia)
- Electricity Particle System and Splitter visuals (Will)
- Electricity and puzzle sounds (Will)

Ending implemented:

- Story implementation (Nathanael/Will)
- End/sword trigger (Olivia)
- Render Texture Downscaling (Will)
- End animation (Olivia/Nathanael)
- Minotaur implementation (Will)
- Sound Design for each step of the end sequence (Will)

All time after these stages are completed will be used for polish.



9. A list of any additional/stretch goals that could be added to the project if time allows.

- Story:
  Lets us explore this world with focus and intentionality. Incorporating this into the larger game will help a lot with fleshing it out.

- Voice/caption implementation:
  There will be a lot of voices in this game, to add most of the story aspect to the game as well as provide advice and help the player complete the game. This will require the recording of voice lines, setting them to trigger and play at certain points during the gameplay, and deciding if they will finish fully or if they can be interrupted. A caption system will display the words that are spoken when they come up, indicate the end of a spoken line, and be able to move to the next line of text early (potentially).
    - Dependencies: Game progress triggers, story, voice lines, caption system
    - Requirements/resources: UW recording studio/editing software, Audacity

- Soundtrack
  Adding a soundtrack to the game would add to the gameplay and help immerse the player in the story. This would require collaboration with someone, or multiple people, who have a background in music. To add the music to the game, triggers would need to be implemented that would signal different songs in the soundtrack to play when the player

reaches certain areas/levels to keep the player immersed and just add to the overall entertainment of the game.

- Dependencies: Game progress triggers, music background experience
- Requirements/resources: Recording studio/editing software, Audacity

- Hint implementation:
  Once puzzles are determined, writing and recording hints will be done using the captions/voice and visuals to help the user progress through the puzzle. These hints must be able to be helpful while not giving away the puzzle and refraining from being too obnoxious. Much playtesting will be required to keep hints helpful to the gameplay and not restrictive.
  - Dependencies: Puzzle Design, Map Design, Story, Assets, full (at least prototyped) system

- Implementation of "Hades" Puzzles
  These puzzles would incorporate the theme of the third major Greek god, Hades. They are themed around the idea of death, with the player needing to interact with death in some way to progress in some way. Initial ideas of this include interacting with ghosts to find solutions to puzzles and drinking poisons to enter a state of near death to see a solution to a puzzle. These would have to be added to the game loop in a nonintrusive way, and it will also require new assets, particles, and systems to be created depending on what systems are made. This may also require adding an extra floor to account for puzzle spacing, which drastically increases the complexity of the game.
  - Dependencies: Assets, Map Design